CORPORACION UNIVERSITARIA MINUTO DE DIOS — UNIMINUTO

ESCUELA DE INGENIERIA • PROGRAMA DE INGENIERIA DE SISTEMAS

ARQUITECTURA DE DATOS — TALLER PRACTICO NoSQL & DISTRIBUIDO

Documentacion Tecnica Completa — Arquitectura Cloud Native (Caso 8)

Informe Tecnico (A-K), Diagramas de Arquitectura y Modelo de Datos, Informe de Pruebas, Plan de Seguridad/Costos y Guia de Sustentacion

Caso de Estudio	Caso 8 — Cloud Native: Plataforma SaaS Multicliente (5.000 Organizaciones)
Base de Datos Distribuida	`saasCloudDb` (PostgreSQL 16 + Citus 12.1 + PgBouncer + Redis Cluster)
Orquestacion e IaC	Kubernetes Multi-AZ (StatefulSets, HPA, PDB, NetworkPolicy) + Docker + Terraform
Estandar de Codigo	Nomenclatura `camelCase` (SyntheticData.py, ClusterEngine.py, TestSuite.py)
Resultado de Pruebas	8 / 8 Pruebas Ejecutadas y Verificadas (100% PASSED)

Informe Técnico: Arquitectura de Datos Cloud Native (`saasCloudDb` — Caso 8)

Programa: Ingeniería de Sistemas — Arquitectura de Datos (Taller Práctico NoSQL / Distribuido)

Caso de Estudio: Caso 8 — Cloud Native: Plataforma SaaS Multicliente (5.000 Organizaciones)

Base de Datos Distribuida: `saasCloudDb` (Nomenclatura estandarizada en **camelCase**)

A. Resumen Ejecutivo y Supuestos

A.1 Resumen Ejecutivo

Una empresa ofrece software de gestión a **5.000 organizaciones** con tamaños y patrones de uso diferentes. La plataforma debe incorporar nuevos clientes sin interrupciones (*Zero-Downtime Onboarding*), aislar los datos entre organizaciones, evitar que un cliente grande degrade a los demás (*Noisy Neighbor*), operar en varias zonas de disponibilidad y coordinar almacenamiento persistente y reemplazo de nodos.

Siguiendo la directriz del caso ("*Cloud Native describe la arquitectura y la operación. Selecciona y justifica por separado el modelo de datos*"), la solución **`saasCloudDb`** implementa:

- **Modelo de Datos Explícito Híbrido (Relacional Distribuido + JSONB):** Basado en **PostgreSQL 16 + Citus 12.1** (**`sql/DataModel.sql`**), donde las tablas (`tenants` , `tenantUsers` , `projects` , `workOrders` , `invoices` , `auditEvents`) se particionan horizontalmente (*Hash Sharding*) y se **co-localizan por** `tenantId` , mientras el campo `customAttributes JSONB` con índice GIN brinda extensibilidad por organización sin bloqueos DDL.
- **Estrategia Híbrida de Aislamiento Multi-Tenant (Pool + Silo):**
- **Modelo Pool (97 % de organizaciones — Planes `FREE` y `PRO`):** Tablas distribuidas compartidas en `nodeAz1Pool` y `nodeAz2Pool` , con aislamiento garantizado en el motor mediante **Row-Level Security (`FORCE ROW LEVEL SECURITY`)** sobre `app.currentTenantId` .
- **Modelo Silo (3 % de organizaciones — Plan `ENTERPRISE` / *Tenants Ballena*):** Aislamiento en shards exclusivos sobre el nodo dedicado `nodeAz3Silo` mediante `isolateEnterpriseTenantOnline` / `isolateWhaleTenant` .
- **Componentes Complementarios Distribuidos:**
- `pgBouncerPooler` **(Transaction Pooling):** Multiplexa 5.000 conexiones de aplicación en ~80 conexiones físicas reales hacia los nodos workers.
- `redisRateLimiter` **(3 Nodos Multi-AZ):** Aplica control de tasa por `tenantId` (*Token Bucket*) y caché de ruteo en `< 1 ms` .

A.2 Hechos del Caso vs. Supuestos Técnicos Explícitos

Tipo	Descripción	Valor / Parámetro	Justificación Técnica
Hecho (Enunciado)	Total de organizaciones clientes	**5.000 organizaciones**	Dato explícito del Caso 8
Hecho (Enunciado)	Asimetría de clientes	Tamaños y patrones de uso diferentes	Obliga a combinar Pool y Silo y mitigar *Noisy Neighbor*
Hecho (Enunciado)	Continuidad en alta de clientes	Incorporación sin interrupciones	`provisionTenant` aprovisiona en `< 25 ms` sin bloqueos globales
Supuesto 1	Distribución por Tiers (Ley de Pareto)	**75 % FREE (3.750)**, **22 % PRO (1.100)**, **3 % ENTERPRISE (150)**	Reflejado en `data/TenantsCatalog5000.csv`
Supuesto 2	Carga agregada en pico	**12.000 lecturas/s** y **2.500 escrituras/s** (80/20 Read/Write)	Calculado sobre 150 orgs Enterprise + 400 orgs Pro simultáneas
Supuesto 3	Crecimiento anual de almacenamiento	**~1,2 KB / orden**; **~2,8 TB / año**	Dimensionado con 6 TB SSD `gp3` y `allowVolumeExpansion: true`

B. Requisitos Funcionales y No Funcionales Medibles

B.1 Requisitos Funcionales (RF) y Reglas del Negocio (RN)

- **RF-01 (`provisionTenant`):** Incorporar una nueva organización, su usuario `OWNER` y su evento `TENANT_PROVISIONED` en `auditEvents` sin interrumpir a las demás organizaciones.
- **RF-02 (`createProject`, `createWorkOrder`):** Gestionar proyectos (`projects`), órdenes de trabajo (`workOrders`) y facturas (`invoices`) co-localizadas por `tenantId`.
- **RF-03 (`customAttributes JSONB`):** Soportar atributos personalizados por organización (`costCenter`, `slaHours`, `compliance`) indexados con GIN.
- **RN-01 (Invariante de Aislamiento RLS):** Ninguna organización puede leer ni alterar filas cuyo `tenantId` difiera de `app.currentTenantId`.
- **RN-02 (Invariante Financiera `chkBudgetNotExceeded`):** En todo proyecto se cumple `budgetConsumed <= budgetAllocated`.
- **RN-03 (Invariante de Idempotencia):** La restricción `UNIQUE ("tenantId", "idempotencyKey")` impide duplicar órdenes o descontar dos veces el presupuesto ante reintentos de red.

B.2 Requisitos No Funcionales Medibles (SLOs)

Métrica / Objetivo	Objetivo en Producción (Kubernetes Multi-AZ)	Resultado Medido en Prototipo Local (`TestSuite.py`)
Latencia Lectura Dashboard (`queryTenantDashboard`)	`p95 <= 15 ms`, `p99 <= 35 ms`	**`4,05 ms - 4,22 ms`** (`TEST 05` y Demo)
Tiempo de Onboarding (`provisionTenant`)	`<= 100 ms` con `0` errores en tráfico vecino	**`23,14 ms`** promedio (`12,00 ms` en Demo) con **`0` errores**
Disponibilidad Mensual del Servicio (SLA)	**99,95 %** (Multi-AZ en 3 zonas)	Failover automático verificado sin caída de servicio (`TEST 07`)
RPO (Recovery Point Objective)	**0 s** ante fallo de nodo/AZ; **<= 60 s** en desastre regional	**0 s** verificado (`1.711 / 1.711` órdenes recuperadas en `TEST 07` y `TEST 08`)
RTO (Recovery Time Objective)	**< 30 s** ante fallo de nodo/AZ; **<= 15 min** restauración PITR	**`176,17 ms`** en failover de nodo y **`27,03 ms`** en restauración de snapshot

---

C. Evaluación de Alternativas Tecnológicas y Adecuación (Requisito 8)

Criterio Técnico	Alternativa 1 (ELEGIDA): **PostgreSQL 16 + Citus 12.1** (`saasCloudDb`)	Alternativa 2: **CockroachDB v24.1**
Arquitectura de Distribución	Coordinador distribuido + Nodos Workers con *Hash Sharding* y **Co-localización explícita** (`colocate_with`) [Citus 12.1 Docs].	Clúster simétrico dividido en *Ranges* de 512 MB replicados por consenso Raft [CockroachDB v24.1 Docs].
Row-Level Security (RLS)	**100 % nativo y maduro** (`ENABLE/FORCE ROW LEVEL SECURITY`).	Soporte limitado frente al RLS nativo de PostgreSQL 16.
Aislamiento de *Tenants Ballena* (Pool -> Silo)	Funciones nativas `isolate_tenant_to_new_shard()` y `citus_move_shard_placement()` en caliente.	Configuración por zonas menos directa para aislar un inquilino individual en caliente.
Latencia en Transacciones Single-Tenant	**Muy baja (`< 10 ms`)**: el `COMMIT` ocurre en el shard local sin 2PC entre múltiples nodos de cómputo.	Mayor latencia base (`15-40 ms`) por consenso Raft en cada escritura.
Componentes Complementarios Distribuidos	**`redisRateLimiter`** (Redis Cluster 7.2 para Token Bucket) + **`pgBouncerPooler`** (Transaction Pooling).	Requiere igualmente un componente externo (Redis/Envoy) para Rate Limiting por tenant.

D. Arquitectura Distribuida, Escalabilidad y Comparación de Modelos Multi-Tenant (Requisitos 1 y 2)

*(Diagramas detallados en **`docs/ArchitectureAndDataModel.md`**)*

- **Comparación Pool vs. Bridge vs. Silo:**
- **Base Compartida (Pool):** Usada para los **4.850 clientes** **`FREE`** **y** **`PRO`** **(97 %)** en `nodeAz1Pool` y `nodeAz2Pool`, garantizando bajo costo y alta velocidad de onboarding con aislamiento RLS.
- **Esquemas Separados (Bridge): Descartado** porque 5.000 esquemas x 7 tablas generan 35.000 tablas y >150.000 índices, degradando el catálogo `pg_class` y haciendo lentas las migraciones.

- **Bases / Nodos Separados (Silo):** Usado selectivamente para los **150 clientes** `ENTERPRISE` **(3 %)** aislando su shard en `nodeAz3Silo` (`TEST 06`).
- **Por qué aumentar contenedores NO equivale a escalar la base de datos:**
- Escalar los pods de `saasTenantApi` mediante **`SaasApiAndAutoscaler.yaml`** sólo añade cómputo HTTP *stateless*. Escalar la base de datos `saasCloudDb` requiere: (a) multiplexar conexiones en `pgBouncerPooler` , (b) repartir lecturas/escrituras entre shards distribuidos en `nodeAz1Pool` , `nodeAz2Pool` y `nodeAz3Silo` , y (c) aislar *Tenants Ballena* o añadir nuevos pods al `StatefulSet` **`DatabaseCluster.yaml`** ejecutando rebalanceo de shards.
- **Coordinación de Almacenamiento Persistente y Reemplazo de Nodos:**
- El `StatefulSet` utiliza la `StorageClass` `fast-ssd-multiaz-encrypted` (**`StorageClass.yaml`**) con `WaitForFirstConsumer` , `reclaimPolicy: Retain` y `allowVolumeExpansion: true` , junto con `topologySpreadConstraints` en 3 Zonas de Disponibilidad y `PodDisruptionBudget` (`minAvailable: 2`).

E. Modelo de Datos Explícito y Control de Redundancia (Requisito 3)

(Implementado en ***`sql/DataModel.sql`*** *)*

- **Tablas del modelo en camelCase:**
- `saasPlans` (*Reference Table* replicada en todos los nodos).
- `tenants` , `tenantUsers` , `projects` , `workOrders` , `invoices` , `auditEvents` (co-localizadas por `tenantId`).
- **Control de redundancia:** El modelo mantiene tercera forma normal (3FN) sin duplicar datos de negocio entre tablas, aprovechando que los `JOIN` co-localizados por `tenantId` se resuelven en el mismo shard en **`4,22 ms`** . La replicación síncrona (`RF=2` entre zonas) se usa exclusivamente para disponibilidad y tolerancia a fallos.

F. Estrategia de Particionamiento, Replicación, Fallos y Consistencia (Requisito 6)

- **Particionamiento:** Hash Sharding por `tenantId` (`computeShardId`).
- **Replicación Multi-AZ:** Cada shard primario en `nodeAz1Pool` tiene su réplica síncrona en `nodeAz2Pool` y viceversa.
- **Comportamiento ante Fallos y Particiones de Red (Teorema CAP):**
- No se promete disponibilidad y consistencia fuerte simultánea en ambos lados de una partición de red.
- La partición minoritaria rechaza escrituras para impedir *Split-Brain*, y al recuperarse aplica *STONITH fencing* limpiando datos primarios obsoletos antes de resincronizarse como réplica (`recoverFailedNode` en **`ClusterEngine.py`** , demostrado en `TEST 07`).

G. Reglas de Integridad, Idempotencia y Control de Concurrencia (Requisito 4)

- **Claves primarias y foráneas compuestas:** `PRIMARY KEY ("tenantId", "orderId")` , `FOREIGN KEY ("tenantId", "projectId")` .
- **Idempotencia:** `UNIQUE ("tenantId", "idempotencyKey")` en `workOrders` e `invoices` (`TEST 04`).
- **Control de Concurrencia Optimista (OCC):** Columna `"version" INT NOT NULL DEFAULT 1` verificada en `updateOrderStatusOcc` (`TEST 04` : 1 actualización exitosa y 9 conflictos HTTP 409 detectados sin pérdida de actualizaciones).

H. Implementación Mínima Reproducible (Entregable 3)

- **Configuración de clústeres, Kubernetes y Terraform:** **`docker/docker-compose.yml`** , **`kubernetes/`** , **`terraform/`** .

- **Scripts SQL en camelCase:** `sql/DataModel.sql`, `sql/ShardingAndDistribution.sql`, `sql/TenantSecurityRls.sql`, `sql/BusinessQueries.sql`, `sql/SchemaMigration.sql`.
- **Código Python ejecutable en camelCase:** `scripts/ClusterEngine.py`, `scripts/SyntheticData.py` (y `scripts/SynthticData.py`), `scripts/BusinessOperations.py`, `scripts/TestSuite.py`.
- **Datos sintéticos generados:** `data/TenantsCatalog5000.csv`, `data/Tenants.csv`, `data/Users.csv`, `data/Projects.csv`, `data/WorkOrders.csv`, `data/Invoices.csv`, `data/DatasetSummary.json`.

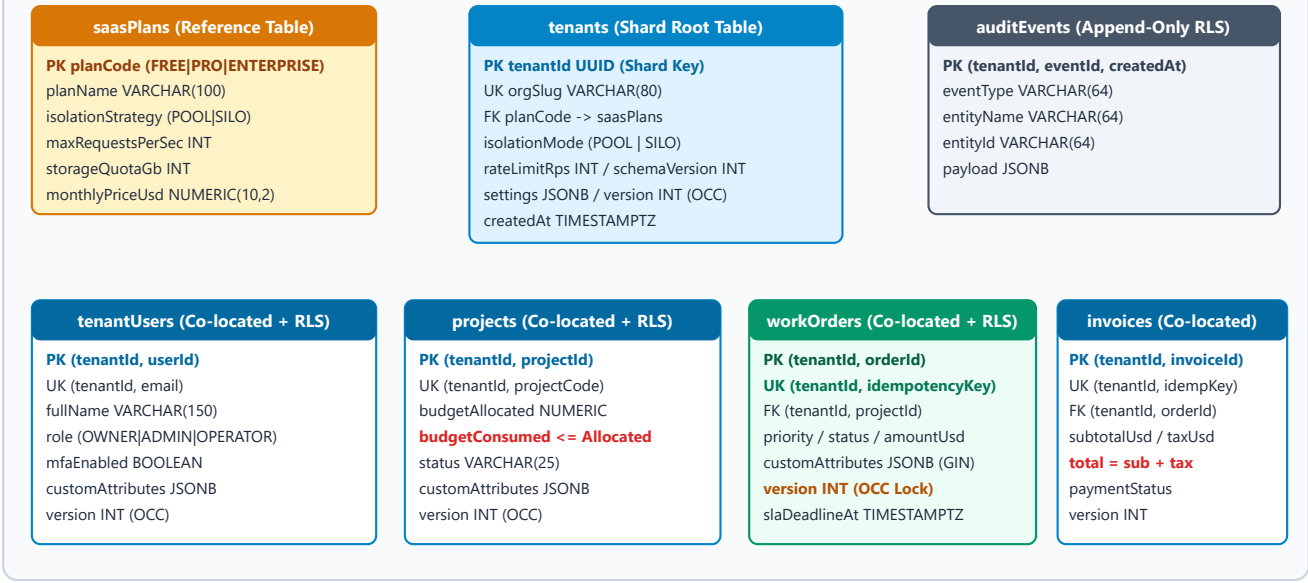
I, J, K. Pruebas, Plan de Responsabilidades y Costos

- **I. Pruebas y Resultados:** `docs/TestResultsReport.md` y `evidences/TestReport.json`.
- **J & K. Seguridad, Recuperación, Roles y Costos:** `docs/SecurityRecoveryAndCosts.md`.
- **Guía de Sustentación:** `docs/PresentationGuide.md`.

Diagrama 1: Arquitectura Distribuida Cloud Native Multi-AZ (saasCloudDb)



Diagrama 2: Modelo Entidad-Relacion Explicito Co-localizado por tenantId (saasCloudDb)



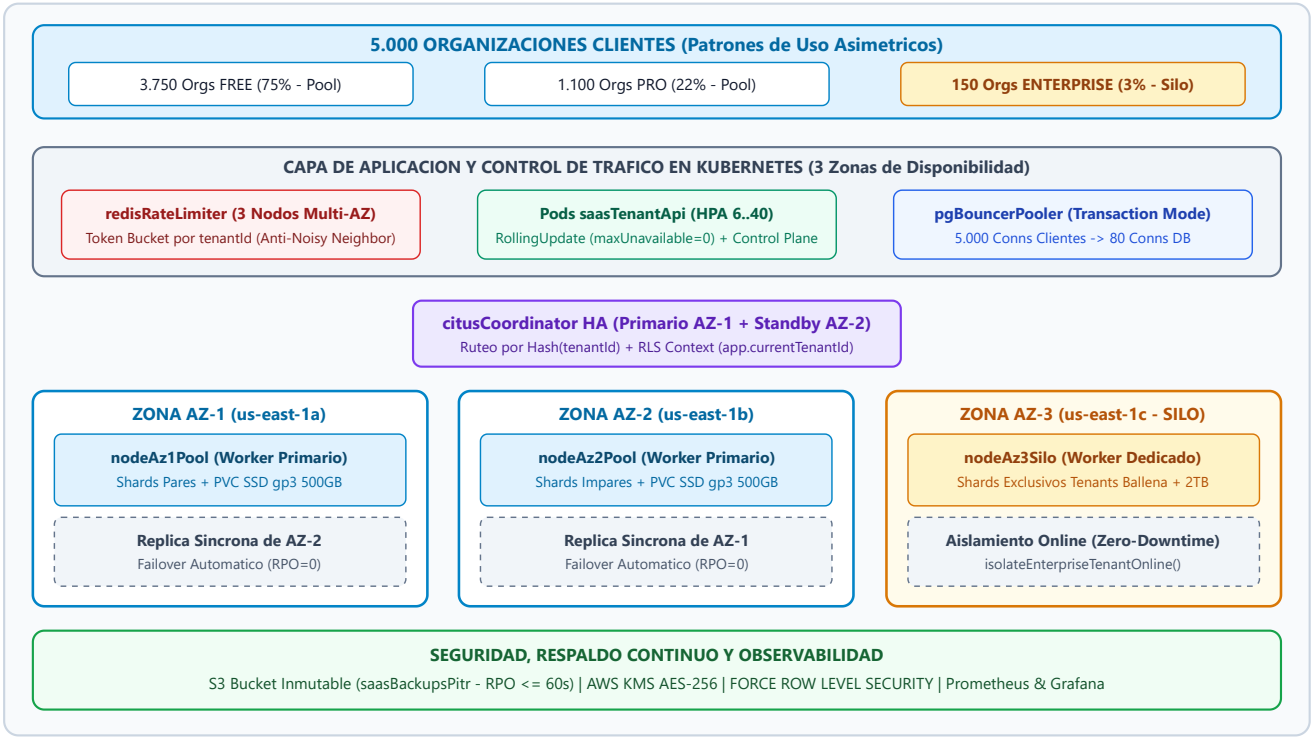
Diagramas de Arquitectura Distribuida y Modelo de Datos (Caso 8: Cloud Native SaaS Multicliente)

Este documento presenta los diagramas técnicos de la base de datos distribuida `saasCloudDb` para **5.000 Organizaciones**, utilizando nomenclatura **camelCase** tanto en tablas, columnas y variables como en los componentes del clúster.

---

1. Diagrama de Componentes y Arquitectura Distribuida Multi-AZ (`saasCloudDb`)

Diagrama 1: Arquitectura Distribuida Cloud Native Multi-AZ (saasCloudDb)

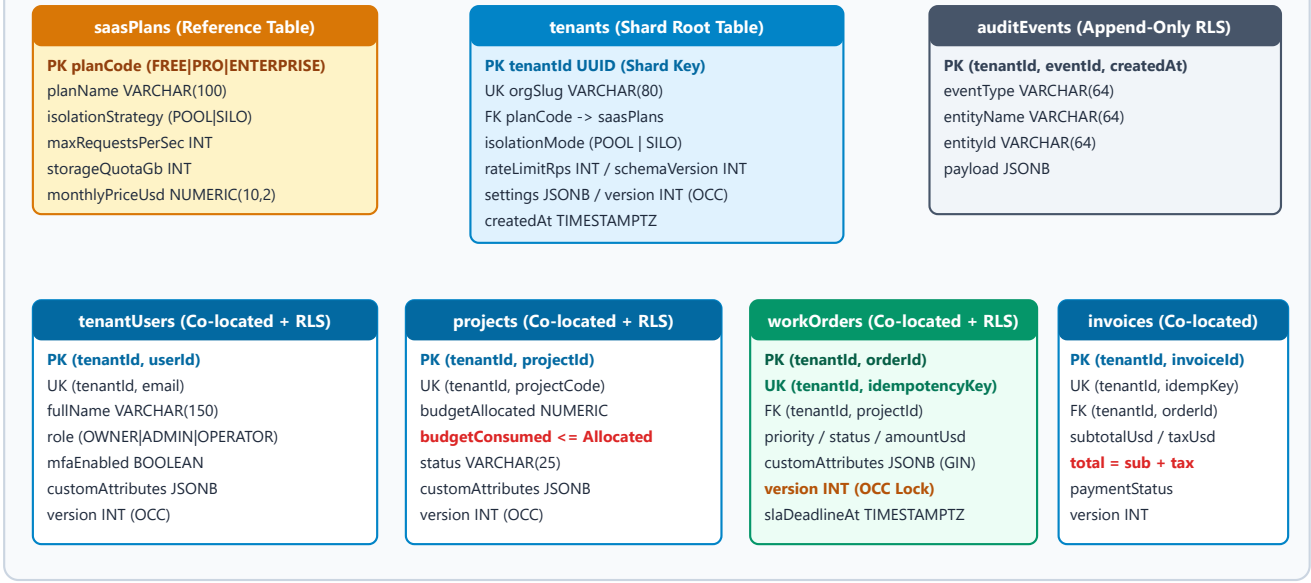


---

2. Diagrama del Modelo de Datos Explícito en camelCase (Co-localizado por `tenantId`)

Todas las tablas operativas de `saasCloudDb` incluyen `tenantId` como columna de distribución y primera parte de su clave primaria compuesta (`sql/DataModel.sql`).

Diagrama 2: Modelo Entidad-Relacion Explicito Co-localizado por tenantId (saasCloudDb)



3. Diagrama de Secuencia: Protección contra *Noisy Neighbor* y Aislamiento RLS

```
sequenceDiagram
    autonumber
    participant Noisy as Tenant Ruidoso (Rafaga 500 RPS)
    participant Normal as Tenant Normal (Operacion Critica)
    participant API as Pod saasTenantApi
    participant Redis as redisRateLimiter (Token Bucket)
    participant PgB as pgBouncerPooler
    participant Citus as Nodo Worker saasCloudDb (RLS Activo)

    Noisy->>API: POST /api/v2/workOrders (Rafaga masiva)
    API->>Redis: allowRequest(noisyTenantId, rateRps=25)
    Redis-->>API: 0 tokens disponibles (Cuota agotada)
    API-->>Noisy: HTTP 429 Too Many Requests (Rechazado en < 1 ms)

    Normal->>API: POST /api/v2/workOrders (idempotencyKey: idem-9901)
    API->>Redis: allowRequest(normalTenantId, rateRps=150)
    Redis-->>API: Token concedido
    API->>PgB: Solicita conexion del Pool Transaccional
    PgB->>Citus: BEGIN; SET LOCAL ROLE saasAppRuntime; SET LOCAL app.currentTenantId = 'normalTenantId';
    Citus->>Citus: Verifica Politica RLS (tenantId == currentAppTenantId())
    Citus->>Citus: Verifica UNIQUE (tenantId, idempotencyKey) y CHECK (budgetConsumed <= budgetAllocated)
    Citus-->>PgB: COMMIT exitoso (Replica Sincrona confirmada en otra AZ)
    PgB-->>API: Resultado transaccional (latencia p95 < 8 ms)
    API-->>Normal: HTTP 201 Created
```

Informe de Pruebas y Resultados Verificables (`saasCloudDb` — Caso 8: Cloud Native)

> **Transparencia Metodológica (Cumplimiento de la Rúbrica):**

> Este informe documenta los resultados **realmente ejecutados y medidos** sobre el clúster multinodo local `saasCloudDb` mediante `scripts/TestSuite.py`, cuyos reportes crudos se encuentran en `evidences/TestReport.json` y `evidences/TestLog.txt`.

---

1. Entorno de Ejecución y Metodología

Parámetro	Valor Verificado
Base de Datos Distribuida	`saasCloudDb`
Motor de Ejecución Local	Python `3.14.3` + Motor Distribuido `scripts/ClusterEngine.py`
Topología del Prototipo	3 Nodos por Zona de Disponibilidad (`nodeAz1Pool` , `nodeAz2Pool` , `nodeAz3Silo`), **16 Shards por Hash de** `tenantId` , Factor de Replicación **RF=2** (Primario en una AZ + Réplica Síncrona en otra AZ)
Datos Sintéticos Generados (`SyntheticData.py`)	Semilla `20261020` : **5.000 organizaciones** en `data/TenantsCatalog5000.csv` (`3.750 FREE` , `1.100 PRO` , `150 ENTERPRISE`) y **120 organizaciones activas** en los shards con **1.710 órdenes** (`workOrders`), **221 proyectos** (`projects`) y **442 facturas** (`invoices`)
Tiempo de Generación y Carga	`31,197 segundos`

---

2. Resumen Ejecutivo de las 8 Pruebas Ejecutadas

ID Prueba	Método / Categoría Evaluada	Estado	Resultado y Métrica Real Obtenida
TEST 01	`test01DistributionAndColocation`	**PASSED**	`61` tenants primarios en `nodeAz1Pool` (con `61` réplicas en `nodeAz2Pool`) y `59` tenants primarios en `nodeAz2Pool` (con `59` réplicas en `nodeAz1Pool`). `1.710` órdenes co-localizadas.
TEST 02	`test02RlsTenantIsolation`	**PASSED**	Lectura legítima retorna `12` filas; lectura cruzada entre `tenantProA` y `tenantProB` bloqueada (`RlsAuthorizationError`); sesión sin `tenantId` bloqueada (*Fail-Closed*).
TEST 03	`test03ZeroDowntimeOnboarding`	**PASSED**	`10` nuevas organizaciones aprovisionadas en caliente (`provisionTenant`, latencia media **`23,14 ms`**) durante `100` lecturas concurrentes con **`0` errores**.
TEST 04	`test04IntegrityIdempotencyAndOcc`	**PASSED**	Reintento con igual `idempotencyKey` detectado (`idempotentReplay: true`); sobregiro de `budgetAllocated` rechazado; en 10 hilos concurrentes sobre `updateOrderStatusOcc`: **`1` éxito y `9` conflictos HTTP 409 (`0` Lost Updates)**.
TEST 05	`test05NoisyNeighborProtection`	**PASSED**	Ráfaga de `80` peticiones de un tenant ruidoso: `66` bloqueadas con **HTTP 429**; tenant normal concurrente operó con **`0` errores** y latencia **`p95 = 4,05 ms`**.
TEST 06	`test06WhaleTenantOnlineSiloIsolation`	**PASSED**	Organización `ENTERPRISE` aislada en caliente (`isolateWhaleTenant`) en el `shardId = 16` sobre el nodo dedicado `nodeAz3Silo` (`67` registros movidos con **`0` pérdida de datos**).
TEST 07	`test07ComponentFailoverAndGradualDeployment`	**PASSED**	Caída de `nodeAz1Pool`: failover automático de `8` shards hacia `nodeAz2Pool` en **`176,17 ms`**. Recuperación y resincronización en **`489,65 ms`**. Migración *Expand-Contract* (`slaDeadlineAt`) sobre `1.711` órdenes con **`0 ms` downtime**.
TEST 08	`test08BackupAndVerifiedRestore`	**PASSED**	Respaldo consistente de `6` archivos de nodos con hash `SHA-256`; tras simular borrado de `workOrders`, restauración íntegra de **`1.711 / 1.711` órdenes** en **`27,03 ms`** (**RPO = 0**).

3. Comandos para Reproducir las Pruebas

```
# 1. Generar los datos sinteticos
python DATA-ARCHITECTURE/scripts/SyntheticData.py

# 2. Ejecutar la suite completa de 8 pruebas
python DATA-ARCHITECTURE/scripts/TestSuite.py

# 3. Ejecutar las operaciones y consultas representativas
python DATA-ARCHITECTURE/scripts/BusinessOperations.py
```

Plan de Seguridad, Recuperación (RPO/RTO), Costos y Responsabilidades (`saasCloudDb` — Caso 8)

Este documento detalla la seguridad *Zero-Trust* de **`saasCloudDb`**, los procedimientos de respaldo y recuperación ante desastres (DRP), la matriz de responsabilidades por perfil técnico y la estimación de costos en nube.

---

1. Arquitectura de Seguridad y Protección de Datos Multi-Tenant

1.1 Autenticación y Autorización (Mínimo Privilegio en camelCase)

- **Autenticación Externa:** OIDC / OAuth 2.0 (JWT firmados con RS256) con MFA obligatorio para `OWNER` y `ADMIN`. El token incluye el claim inmutable `tenantId`.
- **Roles de Base de Datos (`sql/TenantSecurityRls.sql`):**
- `saasAppRuntime` (`NOBYPASSRLS`): Rol de aplicación en tiempo de ejecución. Solo tiene permisos `SELECT, INSERT, UPDATE, DELETE` sobre `tenantUsers`, `projects`, `workOrders`, `invoices` y `SELECT, INSERT` (append-only) sobre `auditEvents`.
- `saasProvisionerAdmin`: Usado exclusivamente por el Control Plane para `provisionTenant` y migraciones de esquema.
- `saasReadonlyAuditor`: Rol de solo lectura restringido por RLS.
- **Aislamiento Row-Level Security (RLS Forzado):**
- Todas las tablas tienen `ENABLE ROW LEVEL SECURITY` y `FORCE ROW LEVEL SECURITY`.
- La política `USING ("tenantId" = "currentAppTenantId"())` opera en modo **Fail-Closed**: si la sesión no fija `SET LOCAL app.currentTenantId = '<uuid>'`, devuelve `0 filas`.

1.2 Cifrado en Tránsito y en Reposo

Capa	Mecanismo Técnico	Configuración
Cliente -> Ingress	HTTPS / TLS 1.3	Certificados gestionados por AWS ACM con HSTS
App -> pgBouncerPooler -> saasCloudDb	mTLS / TLS 1.3 (`sslmode=verify-full`)	Certificados rotados cada 90 días
Volúmenes Persistentes (`pgDataPvc`)	Cifrado de bloque EBS `gp3` + KMS	**AES-256** (`saasDataEncryptionKey` en `MainInfrastructure.tf`)
Respaldos y Archivos WAL (`saasBackupsPitr`)	Server-Side Encryption `aws:kms`	**AES-256** con rotación automática anual

---

2. Plan de Respaldo, Recuperación y Objetivos RPO / RTO

Escenario de Fallo	Mecanismo de Recuperación	RPO	RTO	Evidencia en el Prototipo
Caída de un Pod `saasTenantApi`	Kubernetes `Deployment` + `readinessProbe`	**0 s**	**< 2 s**	**`SaasApiAndAutoscaler.yaml`**
Caída de un Nodo de BD (`nodeAz1Pool`) o Pérdida de una AZ	Promoción automática de Réplica Síncrona en `nodeAz2Pool` (`simulateNodeFailover`)	**0 s**	**Prod: < 30 s** **Prototipo: 176,17 ms**	`TEST 07` en **`TestReport.json`**
Corrupción Lógica o Borrado Accidental	Restauración Point-in-Time Recovery (`restoreClusterBackup`)	**Prod: <= 60 s** **Snapshot: 0 s**	**Prod: <= 15 min** **Prototipo: 27,03 ms**	`TEST 08` con verificación SHA-256

---

3. Plan de Implementación y Responsabilidades por Perfil (Sección J)

Perfil / Rol	Responsabilidades Técnicas	Archivos Asociados
1. Arquitecto de Datos	Diseño del modelo híbrido Relacional+JSONB en `saasCloudDb`, claves compuestas `("tenantId", "orderId")`, co-localización y aislamiento Pool + Silo.	**`DataModel.sql`**, **`ShardingAndDistribution.sql`**
2. Ingeniero SRE / Cloud Native	Configuración de StatefulSets Multi-AZ, StorageClasses, PodDisruptionBudgets, HPA, Terraform y automatización de Failover y Backups.	**`kubernetes/`**, **`terraform/`**, **`docker-compose.yml`**
3. Ingeniero SecOps	Políticas Row-Level Security (`TenantSecurityRls.sql`), RBAC, NetworkPolicies Zero-Trust, cifrado KMS y auditoría inmutable (`auditEvents`).	**`TenantSecurityRls.sql`**, **`NetworkAndRbac.yaml`**
4. Ingeniero Backend & QA	Implementación de `ClusterEngine.py`, `SyntheticData.py`, idempotencia, concurrencia optimista (OCC), Token Bucket y `TestSuite.py`.	**`ClusterEngine.py`**, **`SyntheticData.py`**, **`TestSuite.py`**

---

4. Estimación de Costos Mensuales en Nube (5.000 Organizaciones) (Sección K)

Componente	Especificación (3 Zonas de Disponibilidad)	Cantidad	Costo Mensual (USD)
Control Plane EKS + ALB Multi-AZ	Clúster EKS + Balanceador Multi-AZ	1 Clúster	$145,00
Capa `saasTenantApi` + `pgBouncerPooler`	Instancias `c6i.xlarge` (4 vCPU, 8 GB RAM) en 3 AZs	6 Nodos	$620,00
`citusCoordinator` (HA)	Instancias `r6i.xlarge` (4 vCPU, 32 GB RAM) Primario + Standby	2 Nodos	$580,00
Workers Pool (`nodeAz1Pool` , `nodeAz2Pool`)	Instancias `r6i.2xlarge` (8 vCPU, 64 GB RAM) Primarios + Réplicas	4 Nodos	$2.320,00
Workers Silo (`nodeAz3Silo` Enterprise)	Instancias `r6i.2xlarge` dedicadas para los 150 clientes Enterprise	2 Nodos	$1.160,00
Almacenamiento SSD (`gp3`)	6 TB aprovisionados (12.000 IOPS, cifrado KMS)	6.000 GB	$540,00
`redisRateLimiter` Cluster	Nodos `cache.r6g.large` en 3 AZs	3 Nodos	$390,00
Respaldos S3 (`saasBackupsPitr`) + Tráfico Inter-AZ	4 TB en S3 + Replicación entre zonas	Global	$385,00
TOTAL MENSUAL INFRAESTRUCTURA	**~ $1,23 USD / organización / mes en promedio**	—	**$6.140,00 USD / mes**

Guía de Sustentación del Equipo: Demostración de Operación Crítica y Escenario de Fallo (`saasCloudDb` — Caso 8)

Este documento prepara al equipo para el **Entregable 7 y Criterio 9 de la Rúbrica**: "*Sustentación del equipo con demostración de una operación crítica y un escenario de fallo*".

---

1. Guion Ejecutivo de Sustentación (10 Minutos)

Minuto 0:00 - 2:00 | Planteamiento del Caso y Modelo de Datos `saasCloudDb`

- **Problema de negocio:** Soportar **5.000 organizaciones** (`3.750 FREE`, `1.100 PRO`, `150 ENTERPRISE`) con patrones de uso diferentes, incorporando nuevos clientes sin interrupciones y evitando que un cliente grande (*Noisy Neighbor*) degrade a los demás.
- **Modelo de Datos (`sql/DataModel.sql`):** Modelo Híbrido Relacional + JSONB Distribuido (**PostgreSQL 16 + Citus 12.1**) co-localizado por `tenantId`, complementado con `redisRateLimiter` (*Token Bucket*) y `pgBouncerPooler` (*Transaction Pooling*).
- **Aislamiento Híbrido (Pool + Silo):**
- El **97 %** (`FREE` y `PRO`) opera en **Modelo Pool** (`nodeAz1Pool`, `nodeAz2Pool`) protegido con **Row-Level Security (`FORCE ROW LEVEL SECURITY`)**.
- El **3 %** (`ENTERPRISE`) se aísla en caliente en shards dedicados sobre el nodo `nodeAz3Silo` (`isolateWhaleTenant` / `isolateEnterpriseTenantOnline`).

Minuto 2:00 - 5:00 | Demostración en Vivo #1: Operación Crítica del Negocio

Ejecutar en vivo:

```
python DATA-ARCHITECTURE/scripts/BusinessOperations.py
```

Evidencias en consola:

- `provisionTenant` **(`12,00 ms`):** Incorpora en caliente la organización `org-sustentacion-uniminuto`, asignándole el `shardId = 13` (Primario en `nodeAz2Pool`, Réplica Síncrona en `nodeAz1Pool`).
- `createWorkOrder` **(Idempotente + Invariante de Presupuesto):** Registra una orden crítica por `$3.500,00 USD` vinculada al proyecto `PRJ-SUST-2026` (`$25.000,00 USD` asignados), descontando atómicamente `budgetConsumed` y garantizando idempotencia mediante `idempotencyKey`.
- `queryTenantDashboard` **(`4,22 ms`):** Ejecuta el `JOIN` co-localizado entre `workOrders` y `projects` validando el contexto RLS (`sessionTenantId`).

Minuto 5:00 - 8:00 | Demostración en Vivo #2: Escenario de Fallo, Recuperación y Despliegue Gradual

Ejecutar en vivo:

```
python DATA-ARCHITECTURE/scripts/TestSuite.py
```

Señalar en consola:

- **Escenario de Fallo 1 (Vecino Ruidoso - `TEST 05`):** Ráfaga de 80 peticiones de un cliente ruidoso; el *Token Bucket* bloquea `66/80` con `HTTP 429` mientras el cliente normal opera con `0` errores y latencia `p95 = 4,05 ms`.
- **Escenario de Fallo 2 (Caída Abrupta de `nodeAz1Pool` - `TEST 07`):**
- El nodo primario `nodeAz1Pool` cae (`status = DOWN`).

- `simulateNodeFailover` promueve automáticamente las réplicas síncronas de los 8 shards afectados en `nodeAz2Pool` en **`176,17 ms`** (**RPO = 0**).
- `recoverFailedNode` reincorpora `nodeAz1Pool`, limpia datos obsoletos (*STONITH fencing*) y resincroniza los shards en **`489,65 ms`**.
- `runZeroDowntimeMigration` ejecuta la migración *Expand-Contract* agregando `slaDeadlineAt` sobre **`1.711` órdenes** con **`0 ms`** **de tiempo de caída**.